

Workload Controllers

Deployment · DaemonSet · StatefulSet · kubectl



Deployment



DaemonSet



StatefulSet



Workload Controllers คืออะไร?

ตัวควบคุมที่บอก Kubernetes ว่าควรรัน Pod แบบไหน จำนวนเท่าไร และที่ไหน



Deployment

Stateless

- ▶ สำหรับ Stateless app (web, API)
- ▶ กำหนดจำนวน Replicas
- ▶ Rolling update & Rollback
- ▶ ใช้งานบ่อยที่สุด



DaemonSet

Every Node

- ▶ รัน Pod บน ทุก Node อัตโนมัติ
- ▶ ใช้สำหรับ Agent/Daemon
- ▶ เช่น Log collector, Monitor
- ▶ Node ใหม่ได้ Pod ทันที



StatefulSet

Stateful

- ▶ สำหรับ Stateful app (DB, Cache)
- ▶ Pod มีชื่อและ ID คงที่
- ▶ Persistent Volume ต่อ Pod
- ▶ Deploy/Scale เป็นลำดับ

Deployment — Deep Dive



Rolling Update

อัปเดต Pod ทีละชุด ไม่หยุดบริการ
ควบคุมได้ด้วย maxSurge / maxUnavailable



Rollback

ย้อนกลับด้วย kubectl rollout undo
เก็บ revision history ไว้อัตโนมัติ



Scale

เพิ่ม/ลด replicas แบบ manual
หรือใช้ HPA แบบอัตโนมัติ



Self-healing

ถ้า Pod ตาย Deployment สร้างใหม่ทันที
ให้ครบตามจำนวน desired

YAML ตัวอย่าง

```
apiVersion:
  apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    spec:
      containers:
        - name: web
          image: nginx:1.25
```

DaemonSet — Pod บนทุก Node



Use Cases ที่พบบ่อย



Log Collector

Fluentd, Filebeat



Node Monitor

Prometheus node-exporter



Security Agent

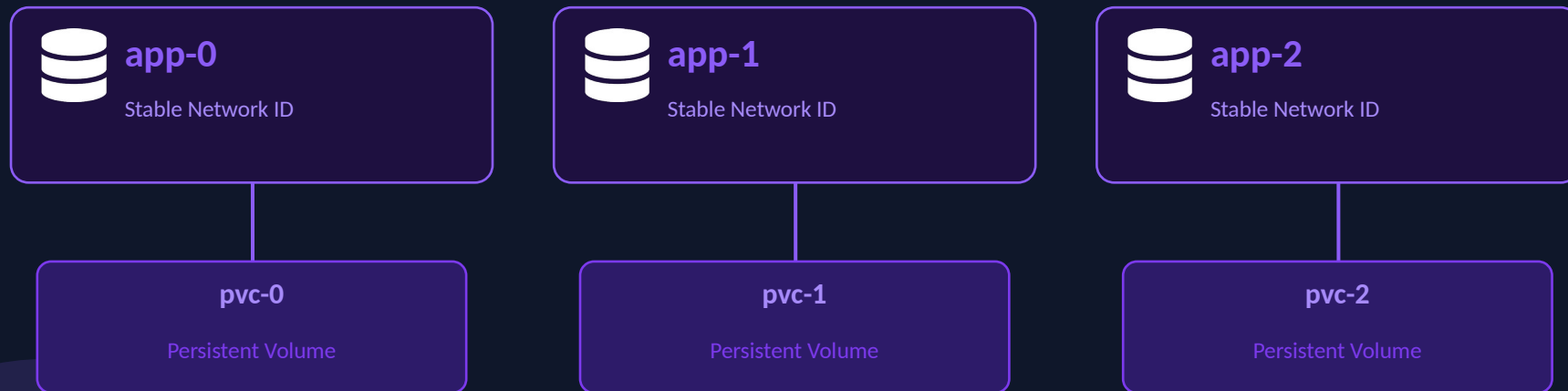
Falco, Sysdig



Network Plugin

Calico, Weave

StatefulSet — Stateful Apps



🏷️ Pod มีชื่อคงที่: app-0, app-1, app-2

💾 แต่ละ Pod มี PersistentVolume ของตัวเอง

1 2
3 4 Deploy & Scale ตามลำดับ (0 → 1 → 2)

🌐 มี Headless Service สำหรับ DNS resolution

เปรียบเทียบ Workload Controllers

คุณสมบัติ	Deployment	DaemonSet	StatefulSet
App ประเภท	Stateless	Agent/Daemon	Stateful
จำนวน Pod	กำหนดเอง	1 ต่อ Node	กำหนดเอง
ชื่อ Pod	สุ่ม hash	สุ่ม hash	คงที่ (0,1,2)
Storage	Shared / None	Shared / None	PVC ต่อ Pod
Scale Order	พร้อมกัน	ตาม Node	เป็นลำดับ
Rolling Update	✓ รองรับ	✓ รองรับ	✓ รองรับ (ลำดับ)
ตัวอย่าง	Web, API	Fluentd, Falco	MySQL, Redis

kubectl — คำสั่งที่จำเป็น

จัดการ Resources

<code>kubectl get deploy</code>	ดู Deployments ทั้งหมด
<code>kubectl get ds</code>	ดู DaemonSets
<code>kubectl get sts</code>	ดู StatefulSets
<code>kubectl get all -n <ns></code>	ทุก resource ใน ns
<code>kubectl describe deploy <name></code>	รายละเอียด Deployment

Scaling

<code>kubectl scale deploy/<n> --replicas=5</code>	Scale Deployment
<code>kubectl scale sts/<n> --replicas=3</code>	Scale StatefulSet
<code>kubectl autoscale deploy/<n></code>	สร้าง HPA
<code>kubectl get hpa</code>	ดู HPA ทั้งหมด
<code>kubectl top pods</code>	ดู CPU/RAM usage

Deploy & Update

<code>kubectl apply -f file.yaml</code>	สร้าง/อัปเดต resource
<code>kubectl set image deploy/<n></code>	เปลี่ยน image version
<code>kubectl rollout status</code>	ดูสถานะการ deploy
<code>kubectl rollout history</code>	ดู revision history
<code>kubectl rollout undo</code>	ย้อนกลับ Deployment

Debug & Inspect

<code>kubectl logs <pod> -f</code>	ดู logs แบบ real-time
<code>kubectl logs <pod> -c <cont></code>	ดู log เฉพาะ container
<code>kubectl exec -it <pod> -- sh</code>	เข้า shell ใน Pod
<code>kubectl get events</code>	ดู Events ใน namespace
<code>kubectl port-forward <pod> 8080:80</code>	Forward port มาที่เครื่อง

kubectl — Context & Namespace



Context Management

เปลี่ยน cluster ที่ใช้งาน

`kubectl config get-contexts` ดู context ทั้งหมด

`kubectl config current-context` context ปัจจุบัน

`kubectl config use-context <name>` เปลี่ยน context

`kubectl config set-context \`

`--current --namespace=<ns>` ตั้ง default namespace



Namespace Commands

แยก environment ภายใน cluster

`kubectl get ns` ดู namespace ทั้งหมด

`kubectl create ns <name>` สร้าง namespace ใหม่

`kubectl get pods -n <ns>` ดู pod ใน namespace

`kubectl get pods -A` ดู pod ทุก namespace

`kubectl delete ns <name>` ลบ namespace

kubectl — Tips & Tricks

Dry Run — ทดสอบก่อน Apply

```
kubectl apply -f deploy.yaml --dry-run=client
```

→ ตรวจสอบ YAML โดยไม่สร้าง resource จริง

Generate YAML จาก Command

```
kubectl create deploy my-app --image=nginx -o yaml --dry-run=client > deploy.yaml
```

→ สร้าง template YAML ได้เลย

Watch แบบ Real-time

```
kubectl get pods -w  
kubectl get pods --watch
```

→ ดูการเปลี่ยนแปลง Pod แบบ real-time

Force Delete Pod ที่ค้าง

```
kubectl delete pod <name> --grace-period=0 --force
```

→ ใช้เมื่อ pod ติด Terminating นาน

Copy File จาก/เข้า Pod

```
kubectl cp <pod>:/path/to/file ./local-file  
kubectl cp ./local-file <pod>:/path/
```

→ คัดลอก file ระหว่างเครื่องกับ Pod

Shorthand Resources

```
po = pods | svc = services  
deploy = deployments | ds = daemonsets  
sts = statefulsets | ns = namespaces
```

→ พิมพ์สั้นลงได้

สรุปบทที่ 2

Deployment

Stateless apps — กำหนด replicas, rolling update, rollback ได้ง่าย

DaemonSet

รัน 1 Pod ต่อ Node — เหมาะกับ log/monitoring agent

StatefulSet

Stateful apps — Pod มีชื่อคงที่, PVC ต่อ Pod, scale ตามลำดับ

kubectl

get, apply, scale, rollout, logs, exec, port-forward